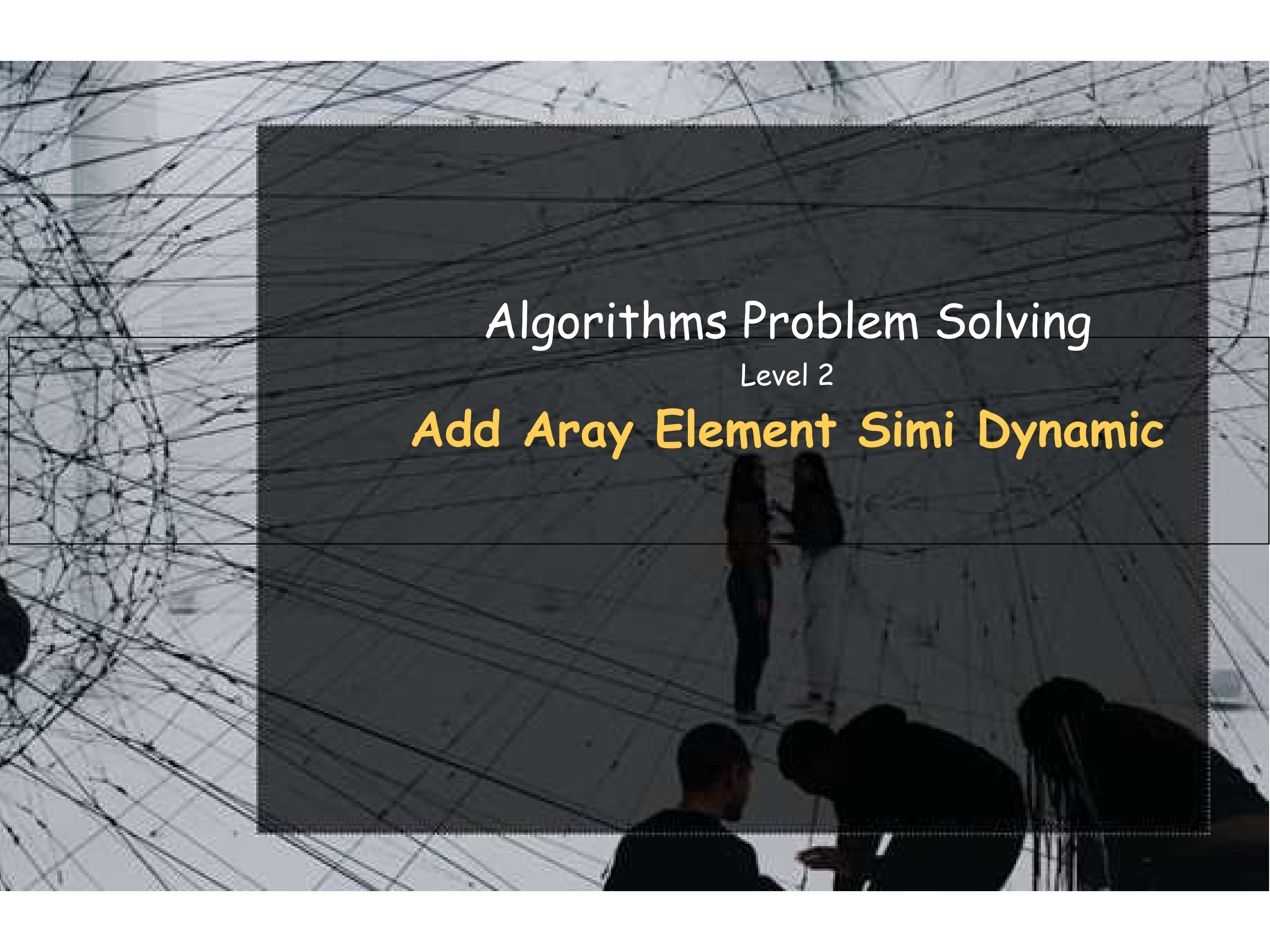


Algorithms Problem Solving

Level 2

Add Array Element Simi
Dynamic



Algorithms Problem Solving

Level 2

Add Array Element Simi Dynamic

Definition

- ◆ This program allows the user to enter numbers one by one and store them in an array.
- ◆ The array has a maximum size of `100`.
- ◆ The actual number of elements is decided during program execution.
- ◆ The user chooses after each number whether to continue adding more numbers.
- ◆ This is called a semi-dynamic array because the array has a fixed maximum size, but the number of used elements changes during runtime.

Explanation

- ◆ The array is created with a fixed maximum size:
`int arr[100];`
- ◆ ``arrLength`` starts at ``0`` because the array initially contains no elements:
`int arr[100], arrLength = 0;`
- ◆ ``AddElementToArray()`` increases the array length first:
`arrayLength++;`
- ◆ Then it stores the number at the new last position:
`arr[arrayLength - 1] = number;`
- ◆ For example, when ``arrayLength`` is ``0``:
`arrayLength++ → 1`
`arr[1 - 1] → arr[0]`
- ◆ When the next number is added:
`arrayLength++ → 2`
`arr[2 - 1] → arr[1]`
- ◆ ``AskUserToInputMoreNumberInArray()`` keeps asking the user for numbers inside a ``do-while`` loop.
- ◆ The loop continues while ``addNumb`` is ``true``.

Steps in main()

1. Create an array with a maximum size of `100`.
2. Set `arrayLength` to `0`.
3. Ask the user to enter a number.
4. Call `AddElementToArray()`.
5. Increase `arrayLength`.
6. Store the number at index `arrayLength - 1`.
7. Ask the user whether they want to add another number.
8. If the user enters `1`, continue.
9. If the user enters `0`, stop.
10. Print the final array length.
11. Print all stored elements.

💡 Example

◆ User enters:

8

20

44

◆ After entering `8`:

arr[0] = 8

arrayLength = 1

◆ After entering `20`:

arr[1] = 20

arrayLength = 2

◆ After entering `44`:

arr[2] = 44

arrayLength = 3

◆ The user then chooses `0` to stop.

➡ Result

Array Length: 3

Array Elements : 8 20 44

✈ Note

- ◆ ``arrayLength`` represents how many elements are currently used, not the maximum size of the array.
- ◆ The physical array size is ``100``.
- ◆ The used part of the array can be smaller than ``100``.
- ◆ ``int& arrayLength`` is passed by reference so ``AddElementToArray()`` can change the original ``arrayLength`` in ``main()``.
- ◆ ``arrayLength - 1`` gives the correct index of the newly added element because array indexes start from ``0``.
- ◆ ``bool addNumb`` receives ``0`` as ``false`` and ``1`` as ``true``.

⊘ Warning

- ◆ The program should not allow more than `100` elements to be added.
- ◆ Currently, there is no check preventing `arrayLength` from becoming greater than `100`.
- ◆ `ReadNumber()` accepts any positive number; it does not limit the entered number to `1-100`.
- ◆ `cmath` and `corecrt\_math\_defines.h` are not needed in this program.
- ◆ The user is expected to enter `0` or `1` for the continue question. Other values can produce unexpected behavior.



Comparison

`arr[100]`

- ✓ Represents the maximum capacity of the array.
- ✓ Can store up to `100` integers.
- ✓ Its size is fixed when the program starts.

`arrayLength`

- ✓ Represents the number of elements currently stored.
- ✓ Starts at `0`.
- ✓ Changes whenever a new element is added.
- ✓ Can range from `0` to `100`.

✦ Summary

- ✓ Create an array with a maximum size of `100`.
- ✓ Start `arrayLength` at `0`.
- ✓ Add elements one by one.
- ✓ Increase `arrayLength` before storing each new element.
- ✓ Store the new element at `arrayLength - 1`.
- ✓ Use a `do-while` loop to ask whether to continue.
- ✓ `arrayLength` represents the number of used elements, not the array's maximum capacity.

Thank you 😊